Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

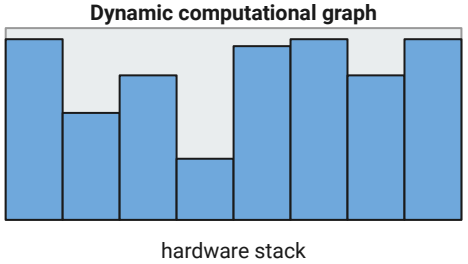
David Raposo1*, Sam Ritter1, Blake Richards1,2, Timothy Lillicrap1, Peter Conway Humphreys1 and Adam Santoro1*
1Google DeepMind, 2McGill University & Mila, *Equal Contribution

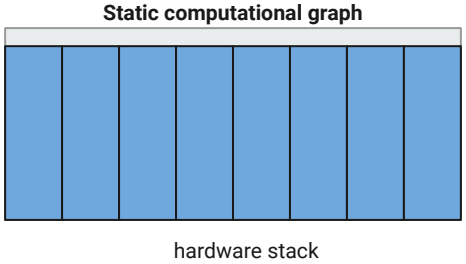
2024.04.35
Juyun Wee

Conditional computation

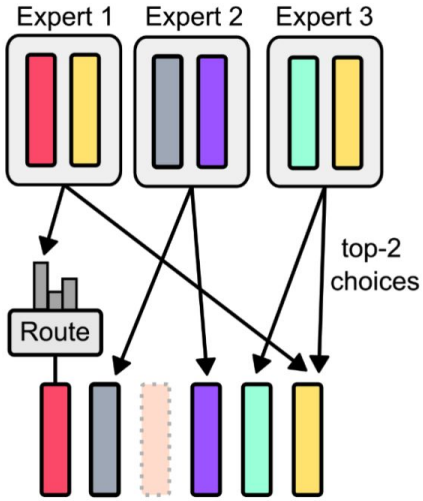
: technique that tries to reduce total compute by expanding it only when needed

- **general formulations of this challenging problem** : may not work well with existing **hardware constraints** since they tend to introduce dynamic computation graphs
- **promising conditional computation methods** : those that are harmonious with our current hardware stack, which prioritizes static computation graphs, and known tensor sizes that are selected to maximize hardware utilization





Mixture-of-Experts (MoE)



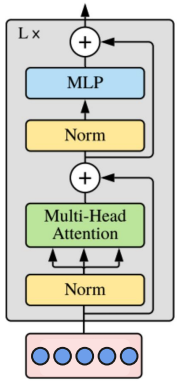
: Mixture of experts (MoE) is a machine learning approach that divides an artificial intelligence (AI) model into separate sub-networks (or “experts”), each specializing in a subset of the input data, to jointly perform a task

- **Distributed Work**: MoE allows individual expert models to be trained and combined separately. This enables parallel processing and distributed learning, making it suitable for large datasets and complex model
- **Improved Performance & Flexibility**

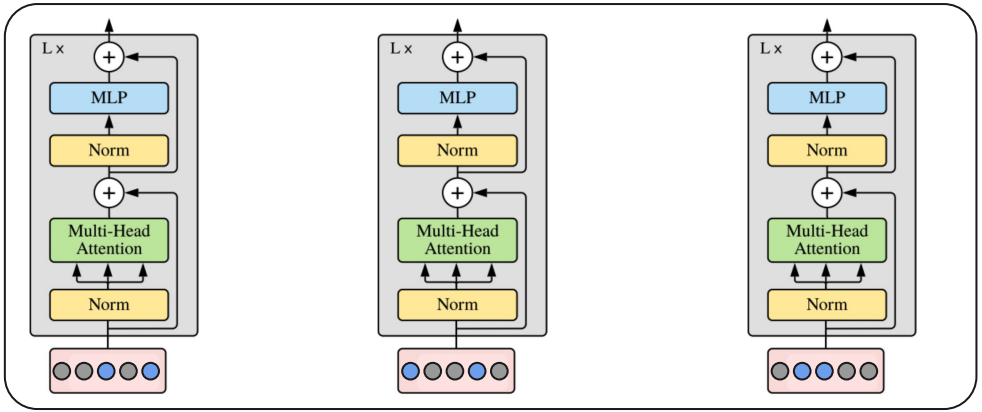
Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

: Dynamically allocate flops according to input sequence

- Optimize sequence allocation by limiting the number of tokens that can participate in calculations at each layer (MLP/ Self-attention)
- Achieving baseline performance within the same flops/training time (dynamic calculation, efficient allocation)

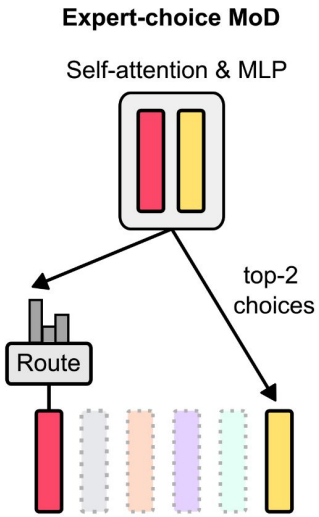


Original Transformer block



Dynamically allocating Flops in each Transformer block

Method Introduction of Mixture-of-Depths



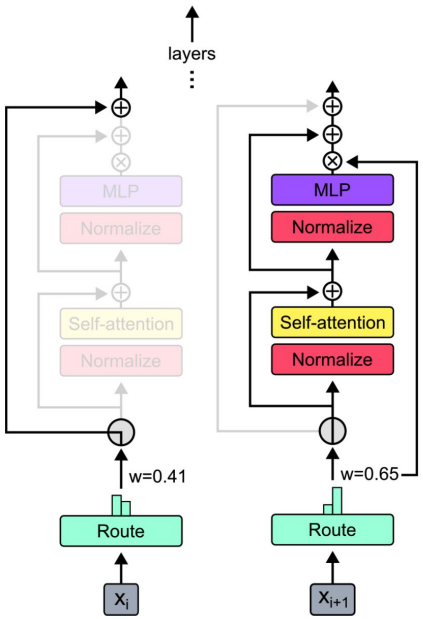
STEP 1 . Select k important tokens

Calculate token importance for each layer (MLP/ Self-attention)

: Only <u>k important tokens</u> for <u>each layer</u> are calculated selectively

- sample token numbers for every layer
- which token is important may differ for each layer)

Method Introduction of Mixture-of-Depths



STEP 2 . Routing tokens

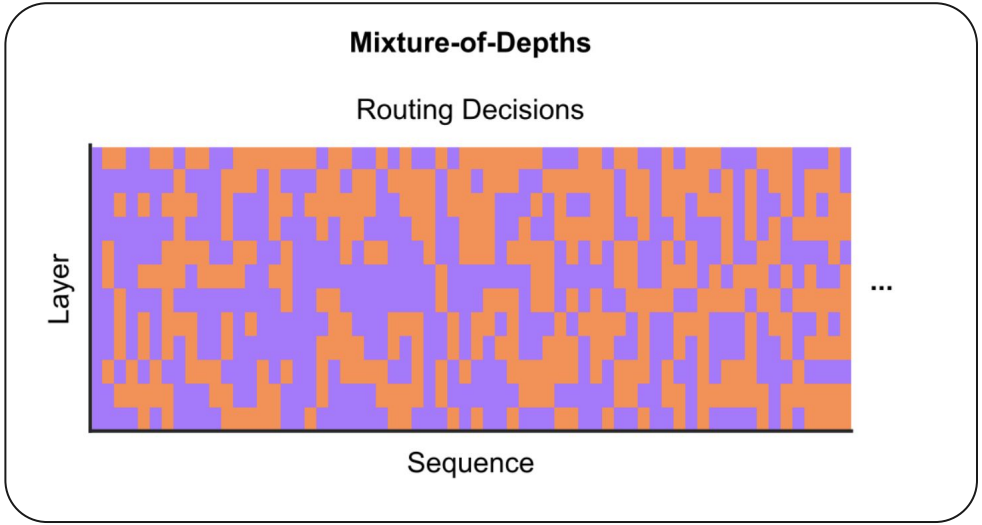
- Route 1: Self-attention/ MLP layer calculation

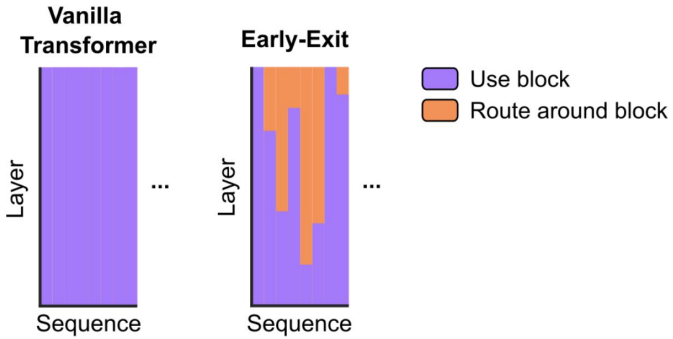
: k tokens those are selected to be important in each layer

- Route 2:Self-attention/ MLP layer calculation

: residual tokens

Method Introduction of Mixture-of-Depths

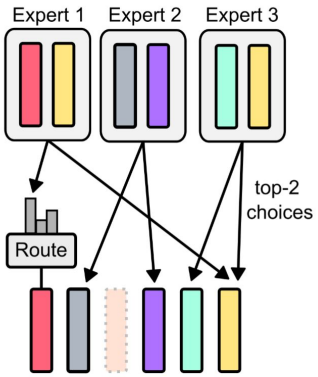




MoE vs MoD

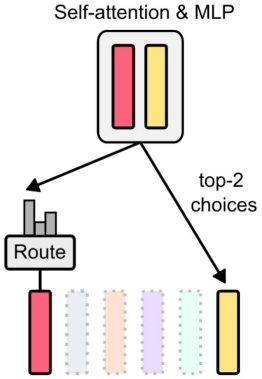
MoE

conditional logic : route tokens to one of many expert MLPs
(keeping total compute expenditure constant)



MoD

routing tokens using the routing logic from MoE transformers, but
rather than having multiple experts, MoD deploys a single expert
(which can be dynamically skipped)



Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

- **set a static compute budget** that is less than that of an equivalent vanilla transformer by limiting the number of tokens n a sequence

- Identify the **<u>top-k scalar weights per block</u>** to select those tokens that will participate in a block's computations
    - The number of tokens to be calculated per rare is set to k
      → The size of the computation graph and tensors remain **static** during the training period.
      A good method from the hardware perspective!

Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

1. Defining a compute budget

- leverage the notion of " capacity"

: it defines the total number of tokens that comprise the input to a given computation

ex. self-attention & MLP : capacity of T

- block's capacity : T → T/2
- query-key matrix multiplication : T → (T/2)^2

- downstream performance will also be affected by how aggressively shrink the blocks capacities

Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

2. Routing schemes

Trial1 : Token routing method

- Stochasticity (akin to layer/ block dropout) → significant acc drop

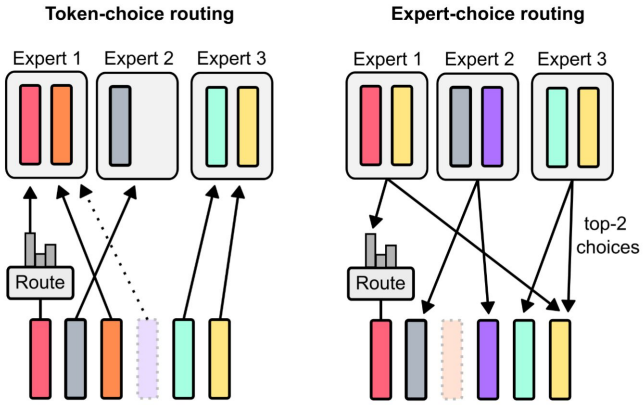
⇒ learned routing is needed!

Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

2. Routing schemes

<Two learned routing schemes>

1) **Token choice**
   : router produces per-token probability distributions across computational paths (e.g., across expert identities in MoE Transformers). Tokens are then shuttled to the path they prefer (that with the highest probability)
   – load balancing problems might occur
2) **Expert choice**
   : rather than having tokens choose the path they prefer, each path instead chooses the top-k tokens based on the tokens' preferences
   - perfect load balance!



Mixture-of-Depths: Dynamically allocating compute in transformer-based language models

2. Routing Implementation

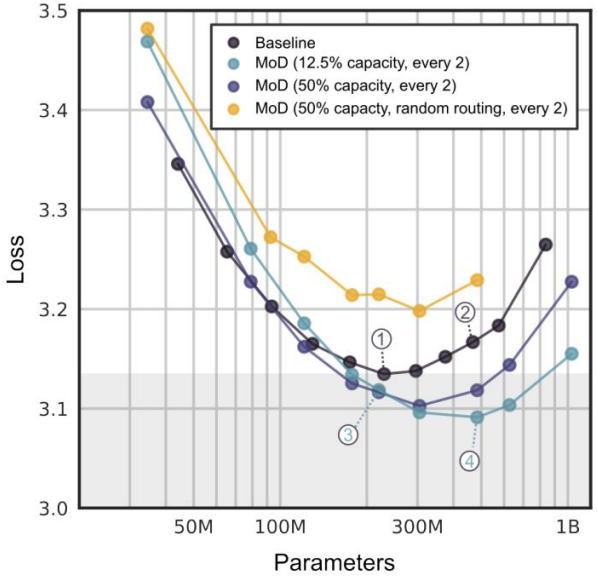
<Two learned routing schemes>

- **set of token embeddings :** $X^l = \{x_i^l | i \text{ is an integer}, 1 \leq i \leq S\}$
- **router weight:** $r_i^l = w_\theta^T x_i^l$

  (scalar produced as a result of a linear projection)
- **β-th percentile of the set of router weights Rl :** $P_\beta(R^l)$

$$x_i^{l+1} = \begin{cases} r_i^l f_i(\tilde{X}^l) + x_i^l, & \text{if } r_i^l > P_\beta(R^l) \\ x_i^l, & \text{if } r_i^l < P_\beta(R^l) \end{cases}$$

Experiments



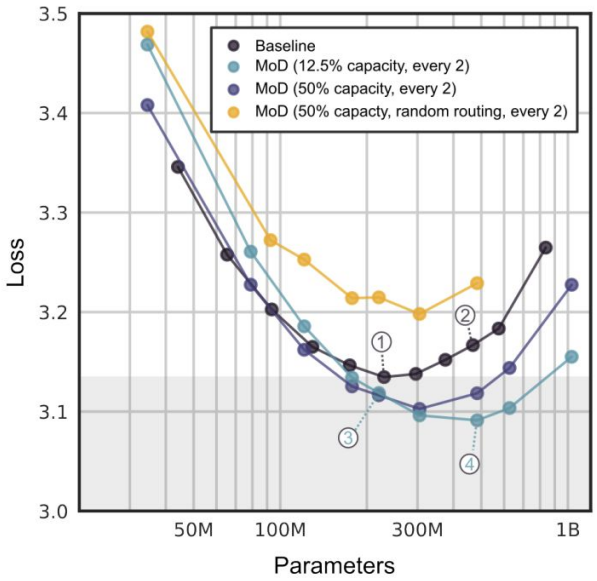
<Settings>
fixed FLOP budget (6e18)

training models with different settings…

- MoD/ Baseline
- number of parameters
- selected k tokens (percentile)

Baseline
MoD (12.5% capacity, every 2)
MoD (50% capacity, every 2)
MoD (50% capacty, random routing, every 2)

Experiments



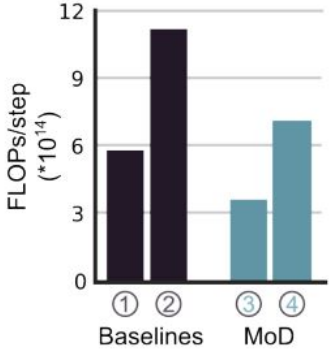
<Setting>
trained models with a relatively small FLOP budget (6e18) to determine optimal hyperparameter k

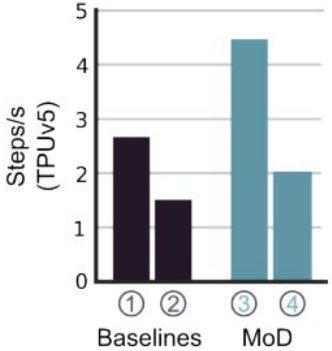
gray box : models that perform better than the optimal baseline (12.5% : calculated, 87.5% : route around)

⇒ optimal MoD transformer achieves a **lower loss** than the optimal baseline as they get more parameters
save FLOPS

: can make the model bigger or training it for longer!

Experiments



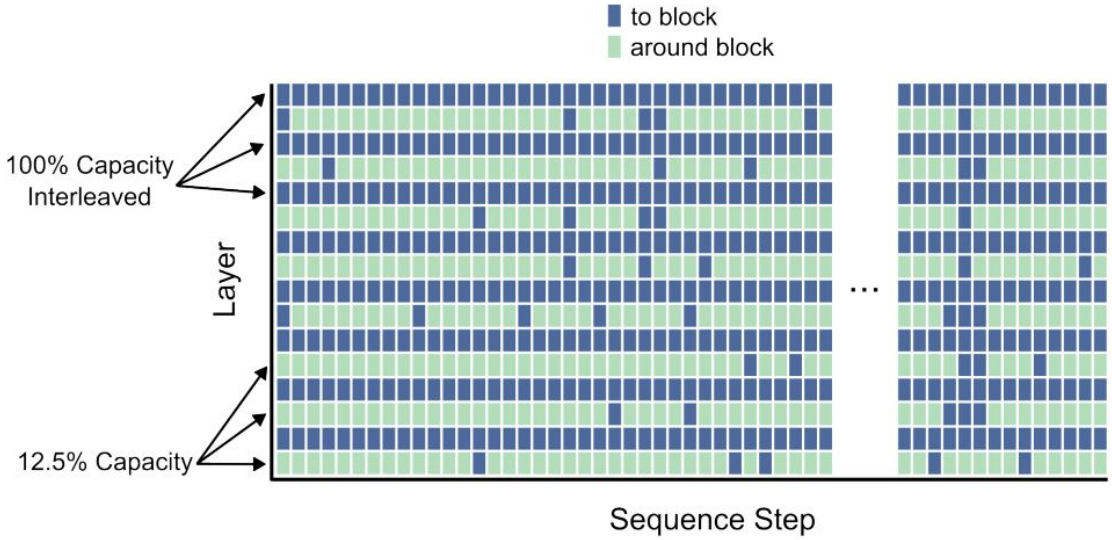


- 66% faster

  :(Due to smaller flops needed for same # of parameters)

- for a given training FLOP budget this method can train models that are both faster and better performing than their baseline counterparts.
  (previously, to train models that are both faster and better performing, the method would likely need more computation)

Experiments



QUESTIONS?